Supplementary Material for "Model-based AI planning enables county-scale farmland consolidation in fragmented mountain landscapes"

Ning Zhou and Xiang Jing

S1 In-house model-free DRL baseline configuration

The Centralised PPO and Multi-Agent Reinforcement Learning (MARL) baselines reported in main-text Section 5 (and used as the fairness anchor for the contrastive-MPC results) were trained in-house under the following protocol.

Environment. Both baselines act on the same Bishan County county-level environment (`CountyLevelEnv`) used by the contrastive MPC pipeline: 13 townships, 2,600 spatial blocks aggregating 52,515 cadastral parcels. The observation is the concatenation of 2,600 blocks $\times$ 17 block-level features plus 12 global features. The action space is a categorical choice over the 2,600 blocks, with the same MaskablePPO-style action mask described in main-text Methods.

Reward. Identical to the main-text reward (Equation 1): $w_S = 4{,}000$, $w_C = 500$, $w_B = 1{,}500$, $b_B = 5$. No method-specific re-tuning was performed, so cross-method comparisons reflect the learning procedure rather than reward engineering.

Centralised PPO. A single MaskablePPO agent with our `ParcelScoringPolicy` architecture (block-wise scoring MLP $[64, 32] \to 1$, global-feature value network). Training: 5 random seeds, 1.5×10^6 environment steps each, $\gamma = 0.995$, $\lambda = 0.95$, learning rate 1×10^{-3} linear-decayed, entropy coefficient 0.01, clip range 0.2, batch size 4,096, mini-batch size 256, 10 epochs per update. Wall-time: 8–12 GPU-hours per seed on a single A100. Cross-seed slope improvement: $-0.79\% \pm 0.36\%$, with 2/5 seeds collapsing into degenerate trajectories.

Multi-Agent Reinforcement Learning (MARL). A 13-agent variant in which each township is a separate MaskablePPO agent with shared parameters and a centralised value function over the global state, trained jointly. Same hyperparameters as Centralised PPO. 5 seeds, all stable. Cross-seed slope improvement: $-0.81\% \pm 0.10\%$ (CV $\approx 12\%$).

Why these are the relevant baselines. Both baselines are task-faithful: same environment, same action space, same reward function, same evaluation protocol (5-episode deterministic rollout) as the contrastive MPC pipeline. The wall-time and seed-failure characteristics motivate the search for a less compute-intensive learned-model alternative; the slope-improvement magnitudes (-0.8% range) provide a non-trivial reference against which the contrastive-MPC result ($-1.289\% \pm 0.079\%$ on Bishan) is then compared.

S2 MPC optimisation ablation on the MSE-only ensemble (Supplementary Table S1)

Table S1 below reports the full MPC optimisation ablation cited in main-text Section 3 (paragraph beginning "Four independent operational failures"). All variants use the same MSE-only world-model ensemble (Bishan District, 5-episode evaluation) and modify a single dimension of the planner. None exceeds the random-sampling baseline; the catastrophic failure of greedy continuation under the MSE-only ensemble (slope +0.029%, wrong direction) is the variant that contrastive training subsequently rescues into the strongest variant tested (main-text Table 3).

Table S1: MPC optimisation ablation across four dimensions, on the MSE-only world-model ensemble (Bishan District). All variants start from the optimal H=5, K=50 random-reward configuration and modify one dimension. None exceeds the random-sampling baseline; aggressive exploitation (greedy continuation) actively reverses the slope direction. The diagnostic shows that the failure is structural to the learned model rather than to the search procedure.

Dimension	Configuration	Slope %	Reward	Diagnosis
Continuation	Random (baseline)	-0.209 ± 0.008	-18.52	—
	Greedy	$+0.029 \pm 0.005$	-95.29	catastrophic
Scoring	Reward (baseline)	-0.209 ± 0.008	-18.52	—
	Slope-only	-0.001	-98.95	zero discrimination
Terminal value	No value (baseline)	-0.209 ± 0.008	-18.52	—
	Value-guided (v=1.0)	-0.172 ± 0.042	-27.96	no improvement
Search budget	K=50 (baseline)	-0.209 ± 0.008	-18.52	—
	K=100	-0.167 ± 0.053	-8.94	no improvement
	K=50, n_r=3	-0.171 ± 0.046	-28.33	no improvement

S3 Synthetic-benchmark secondary metrics (Tables S2–S4)

Main-text Table 5 (`tab:bench-slope`) reports the slope-improvement profile across the seven synthetic landscapes. The three secondary metrics referenced in main-text Section 6 (Δcontiguity, Δ*baimu fang* count, and Δ*baimu fang* area, all under the same 50-pair swap budget and n=5 seeds per cell) are tabulated here. Together with main-text Table 5 these summarise the full $5\times7\times5 = 175$-cell sweep. Aggregation script: `scripts/si_tables_v7.py` in the released code repository.

S4 Toolchain pre-flight checks

Tool 5 of the ArcGIS Pro toolchain (main-text Section 7) validates the user environment before any planning step is allowed to run. The complete list of checks is:

1. **Python interpreter**: ArcGIS Pro 3.6+ `arcgispro-py3` or compatible Python 3.10+ environment.
2. **Spatial Analyst extension**: licence checked out (required by Tool 1 for slope derivation).

Table S2: Synthetic-benchmark contiguity change (ΔContiguity, absolute units). Mean ± standard deviation across n=5 seeds. Higher (more positive) is better; per-row best in bold.

Preset (n_{blocks})	Random	Greedy	GA	PPO	MPC
`bishan_clone` (2,600)	0.0016±0.0007	0.0009±0.0006	**0.0253±0.0023**	0.0027±0.0005	0.0032±0.0009
`neijiang_clone` (3,700)	0.0014±0.0004	0.0012±0.0001	**0.0159±0.0005**	0.0015±0.0004	0.0023±0.0005
`plain_small_cons` (800)	0.0057±0.0023	0.0036±0.0010	0.0018±0.0057	**0.0069±0.0014**	0.0068±0.0015
`plain_large_cons` (4,500)	0.0010±0.0001	0.0008±0.0002	0.0010±0.0003	**0.0014±0.0002**	0.0012±0.0003
`plain_medium_frag` (2,600)	0.0018±0.0004	0.0018±0.0005	0.0005±0.0006	0.0022±0.0003	**0.0026±0.0004**
`mixed_medium_frag` (2,600)	0.0022±0.0006	0.0018±0.0003	**0.0059±0.0016**	0.0031±0.0007	0.0032±0.0008
`hilly_small_cons` (800)	0.0058±0.0020	0.0067±0.0017	**0.0734±0.0055**	0.0092±0.0020	0.0095±0.0013

Table S3: Synthetic-benchmark large-patch count change (Δ*baimu fang* count, ≥ 6.67 ha qualifying patches gained per episode). Mean ± standard deviation across n=5 seeds. Higher is better; per-row best in bold.

Preset (n_{blocks})	Random	Greedy	GA	PPO	MPC
`bishan_clone` (2,600)	0.0±2.2	−0.2±2.6	−3.8±4.4	**9.4±2.6**	9.2±2.3
`neijiang_clone` (3,700)	0.6±0.9	−1.4±1.5	−4.4±3.2	**6.4±2.1**	4.6±3.4
`plain_small_cons` (800)	0.0±0.0	0.2±1.1	**0.8±2.0**	−0.4±2.5	0.4±2.1
`plain_large_cons` (4,500)	0.0±0.0	0.0±0.0	0.2±0.4	**0.8±0.8**	0.0±0.7
`plain_medium_frag` (2,600)	0.6±0.9	−0.6±1.3	1.4±1.8	**1.6±1.1**	0.4±1.3
`mixed_medium_frag` (2,600)	−0.2±1.3	0.0±1.2	**1.2±2.2**	−1.0±2.2	−2.4±1.1
`hilly_small_cons` (800)	−1.0±0.7	−0.4±0.9	−3.2±2.2	3.4±1.7	**9.8±1.5**

Table S4: Synthetic-benchmark large-patch area change (Δ*baimu fang* area, ha). Mean ± standard deviation across n=5 seeds. The sign of this metric is morphology-dependent (see main-text Section 5 discussion of the Bishan vs. Neijiang reversal); per-row best (most positive) in bold for reference.

Preset (n_{blocks})	Random	Greedy	GA	PPO	MPC
`bishan_clone` (2,600)	9.8±11.1	11.9±15.0	155.3±40.3	134.7±26.1	**177.5±31.7**
`neijiang_clone` (3,700)	15.2±9.2	27.8±16.4	**190.8±33.2**	122.4±44.8	167.6±23.0
`plain_small_cons` (800)	20.2±8.9	14.7±16.2	11.5±19.5	19.0±25.3	**30.5±17.5**
`plain_large_cons` (4,500)	9.2±8.9	39.7±13.3	12.8±13.4	40.1±55.3	**40.3±15.3**
`plain_medium_frag` (2,600)	22.8±10.5	38.5±22.8	34.2±34.3	**54.0±40.1**	52.8±26.8
`mixed_medium_frag` (2,600)	14.2±7.3	28.4±13.6	60.7±24.8	**106.8±24.2**	102.1±13.1
`hilly_small_cons` (800)	21.6±14.2	4.6±13.9	**167.7±41.2**	100.3±30.3	131.9±43.5

3. **Required packages**: `gymnasium` (≥ 0.29), `libpysal` (≥ 4.7), `onnxruntime` (≥ 1.16), `numpy`, `pandas`, `scipy`.
4. **ONNX-ensemble shape compatibility**: if a pre-trained ensemble is provided, the baked-in N_{blocks} matches the prepared dataset.
5. **Coordinate reference system**: the input shapefile's CRS resolves to a metric projection (default `EPSG:32648`; user-overridable).
6. **Disk space**: at least 5 GB free in the prepared-data directory.

Tool 5 prints an `[OK]` or `[FAIL]` verdict for each item and refuses to proceed with downstream tools if any required item fails. The 70-second smoke test on the bundled 2-township subset of Neijiang Dongxing District exercises every check end to end and is the recommended onboarding action for new users of the toolchain.

S5 Data and code release

The synthetic benchmark, the pre-trained Bishan ensemble, the ArcGIS Pro toolbox source, and the aggregation/sweep scripts referenced throughout this paper are released under the licences indicated in the main-text Data and Code Availability statements. Repository structure at the public release:

- `LandUseOptimization_P9.pyt`: ArcGIS Pro Python Toolbox manifest.
- `farmland_mpc/`: pure-Python core (loss functions, ensemble training, MPC planner, ONNX export).
- `benchmark/`: synthetic generator, presets, baseline runners, sweep manifest, and the 175-cell results.
- `docs/BENCHMARK_RESULTS.md`: per-(preset, method) aggregated results derived from `benchmark/results/`.
- `scripts/aggregate_results.py`, `scripts/si_tables_v7.py`: aggregation and SI-table generators.
- `environment.yml`, Colab demo notebook, integration smoke test.

S6 Verification stack: full per-episode data (Tables S5–S8)

The main-text Methods sections (`Independent GIS-grounded verification`, `Direct measurement of ensemble 1-step prediction accuracy`, and `Replacing the ensemble with the true environment`) summarise four of the five independent verification layers underwriting the Bishan headline result (the fifth, Neijiang cross-county replication, is in main-text Section 5). Tables S5–S8 give the per-replicate, per-channel numbers for each verification layer so external readers can audit them without rerunning the released scripts.

Layer (i): GIS-grounded recomputation. Table S5 compares the planner-reported deltas (from `mpc_summary.json`) against an independent recomputation that reads only the optimised DLTB shapefile, joins per-parcel slope from the prepared DEM raster, builds Queen contiguity via `libpysal`, and applies the same metric definitions as the main-text Methods. The recomputation script (`validate_optimized_shp.py`) shares no source with the optimisation pipeline.

Table S5: Independent GIS recomputation versus planner-reported deltas, Bishan run_1, 53,086 swappable parcels. The recomputation invokes no learned model. Differences are at floating-point reduction noise level.

Quantity	Recomputed (independent)	Planner-reported	$\lvert\Delta\rvert$
Δ slope (%)	-2.0000	-2.0006	6×10^{-4} pp
Δ contiguity	$+0.012739$	$+0.012748$	8×10^{-6}
Δ baimu count	$+8$	$+8$	0 (exact)
Δ baimu area (ha)	-473.2950	-473.2950	$< 10^{-4}$ ha
farm→forest swaps	428	428	0 (exact)
forest→farm swaps	428	428	0 (exact)

Layer (ii): ensemble member-subset ablation. Table S6 reports the three replicates that constitute the variance estimator used in the main-text Methods caveat on five-seed determinism. Each replicate runs the full MPC pipeline (H=5, K=50, greedy continuation, scoring=reward) with a different 2-of-3 ensemble member subset. The full ensemble's result lies inside the mean $\pm$ sd of the subset distribution on every reported metric.

Table S6: Per-replicate results for the member-subset ablation. The three replicates exhaust C(3,2)=3 distinct subsets. Full-ensemble values reproduced from main-text Table 4 / `mpc_summary.json` for comparison; they fall inside the subset mean$\pm$sd on every metric.

Replicate (members)	Δ slope (%)	Δ cont	Δ baimu count	Δ baimu (ha)
{0, 1}	-2.0222	$+0.01361$	$+5$	-475.02
{0, 2}	-1.9706	$+0.01437$	$+5$	-443.12
{1, 2}	-2.0024	$+0.01128$	$+6$	-505.39
Mean $\pm$ sd (n=3)	-1.9984 ± 0.0260	$+0.01309 \pm 0.00161$	$+5.33 \pm 0.58$	-474.51 ± 31.14
Full ensemble {0,1,2}	-2.0006	$+0.01275$	$+8$	-473.30

Layer (iii): direct ensemble 1-step accuracy. Table S7 reports the per-channel prediction error of the deployed 3-member ensemble against the true environment, accumulated over the 100 in-distribution steps that reproduce episode 0 of `mpc_summary.json`. The reward head is strongly under-fit but Spearman rank correlation between predicted and true reward is positive, and slope-channel error is sub-variance.

Layer (iv): true-environment MPC counter-factual. Table S8 compares the ensemble-MPC headline trajectory against an ablation that replaces every ensemble call with the actual env.step, under matched H/K/γ and a single seed. Implementation concessions (random rather than greedy continuation; stage-1 subsample of 200 of $\sim$2,000 valid actions) are forced by the cost of incremental simulation. Despite using a perfect dynamics model, the resulting trajectory is strictly worse on all three objectives.

Per-step trajectory matching between layer (iii) and the deployed pipeline is byte-identical at every monitored step (10, 20, ..., 100) of episode 0; this is the most stringent reproducibility check the verification stack supports and is therefore the canonical evidence that the deployed ensemble's planning is deterministic given a fixed prepared dataset and config.

Table S7: Ensemble 1-step prediction error over the same 100-step trajectory MPC visits (Bishan, seed 0). True-reward statistics are computed from the actual env.step return. Spearman rank correlation between predicted and true reward is +0.216. Pearson correlation between ensemble disagreement r_{std} and absolute prediction error is +0.077, i.e. disagreement does not flag where members are wrong.

Quantity	Value
Reward MAE	0.818
Reward RMSE	1.540
Reward median |err|	0.170
Reward p90 |err|	3.148
Reward p99 |err|	5.075
r_{true} range / std	$[-3.04, +6.36]$ / 1.397
r_{pred} range / std	$[+0.43, +0.67]$ / 0.069
Naive constant-mean baseline MAE	0.828
Spearman $(r_{\text{pred}}, r_{\text{true}})$	+0.216
Ensemble r_{std} mean / median / max	0.027 / 0.030 / 0.075
Pearson $(r_{\text{std}}, \lvert\text{err}\rvert)$	+0.077
gf[4] slope-improvement MAE	0.00238 (channel std 0.00521)
gf[6] baimu-count-norm MAE	0.00270 (channel std 0.00817)

Table S8: True-environment MPC vs. deployed ensemble MPC, Bishan, 100 outer steps, seed 0. "True-env" replaces every ensemble call with actual env.step; otherwise H=5, K=50, γ=0.99 unchanged. Implementation concessions stated in caption: random (rather than greedy) continuation, stage-1 subsample of 200 valid actions per step.

Quantity	Ensemble MPC (run_1)	True-env MPC	Δ (true − ensemble)
Δ slope (%)	−2.0006	−1.8187	+0.182 pp (worse)
Δ contiguity	+0.01275	+0.00865	−0.0041 (worse)
Δ baimu count	+8	+2	−6 (worse)
Δ baimu area (ha)	−473.30	−565.10	−91.8 ha more loss
Wall time (excl. env build)	1,448 s	358 s	true-env 4× faster
Stage-1 coverage	all valid actions (batched)	200-action random subset	—
Stage-2 continuation	greedy	random	—